진행 중 · 2026.09 -

ROS 2 기반 스마트팜 자동화

예찰·수확·운송·선별을 연계하는 다중 로봇 협업 시스템

파프리카 농작업 자동화를 목표로 예찰·수확·운송 로봇과 선별 장치를 연계하는 프로젝트입니다. 중앙 관제에서 작업을 배정하고 상태를 관리하며, 예찰 결과를 수확·운송·선별 공정으로 연결합니다.

담당 역할

예찰 로봇 주행·수확 작업 연계

예찰 로봇의 자율주행·복귀·도킹과 수확 작업 데이터 설계를 담당합니다. AI 탐지 결과를 관제 및 수확 로봇과 연계하는 임무 제어 구조를 설계·개발하고 있습니다.

설계·개발 범위

- **예찰 경로 주행 · 복귀 · 도킹**
  Nav2 경로 주행과 스테이션 복귀·도킹을 예찰 임무의 이동 기능으로 구성합니다.
- **수확 작업 위치·데이터 정의**
  AI 탐지 결과와 로봇 위치를 기반으로 작업 범위·기준 위치·순서·예상 수확량을 정의합니다.
- **FSM · 관제 인터페이스 설계**
  FSM의 상태 전이와 Action·Topic을 정의해 임무 요청, 진행 보고, 상태 전달 구조를 설계했습니다.



스마트팜 재배 환경 모델

시스템 구성

예찰·수확·운송·선별 공정 연계

예찰 데이터 기반 작업 배정
수확물 인계 및 후속 공정 연동



실물 로봇 및 재배 구역 테스트 환경

01 **예찰** 담당 · 경로 주행 및 작업 생성	02 **수확** 로봇팔 기반 작물 수확	03 **운송** 수확물 이송	04 **선별** 수확물 분류

ROS 2 · Nav2 · FSM · Action / Topic · 7인 팀

예찰 로봇 주행·수확 작업 연계 설계

Nav2 경로 주행, AI 탐지 기반 수확 작업 생성, 관제 통신을 연결하는 예찰 임무 구조

예찰 임무 처리 흐름

01 Nav2 기반 예찰 경로 주행

관제에서 수신한 목표 위치를 Nav2에 전달하고, 재배 구역별 경로를 순차 주행하도록 설계했습니다.

02 탐지 위치 기반 수확 작업 범위 산정

최초 탐지 시점의 로봇 위치, 주행 방향, 로봇팔 유효 작업 폭으로 Harvest Window의 시작·끝과 기준 작업 위치를 계산하는 구조를 정의했습니다.

03 수확 작업 데이터 생성

기준 위치·작업 순서·예상 수확량을 Harvest Task로 구성하도록 정의했습니다. 예상 수확량이 0인 구간은 작업을 생성하지 않고 검사 완료만 보고합니다.

04 진행 보고 · 복귀 임무 분리

구간·구역 완료는 Feedback, 전체 예찰 종료는 Result로 구분했습니다. 스테이션 복귀·도킹은 별도 Action으로 정의했습니다.

수확 작업 생성 구조



FSM 상태별 처리 책임 정의

예찰: 대기 → 주행 → 작업 범위 확정 → 수확량 확정 → 구역 완료

복귀: 스테이션 이동 → 접근·도킹 → 대기

ROS 2 인터페이스 설계

Action으로 예찰·복귀 임무를 요청하고, Topic으로 로봇 위치와 상태를 지속 보고하도록 통신 역할을 구분했습니다.

개발 진행 현황

설계 산출물 · 요구사항, FSM, 수확 작업 데이터, ROS 2 인터페이스 초안

구현·검증 범위 · 경로 주행, AI 연동, 작업 생성, 복귀·도킹

2026.09.17 기준 · 성능 지표 및 주행·도킹 파라미터는 통합 검증 후 확정 예정

POCO · 온디바이스 AI 자세 코칭

자세 분석과 4축 모니터암 제어를 결합한 작업환경 조절 시스템

Raspberry Pi에서 사용자 자세를 분석하고 모니터의 위치와 기울기를 조절하는 시스템입니다. 자세 인식, 모니터암 제어, 사용자 보정, 측정 기록과 리포트를 하나의 장치로 통합했습니다.

담당 역할

짐벌 · 프론트엔드 · 멀티프로세싱

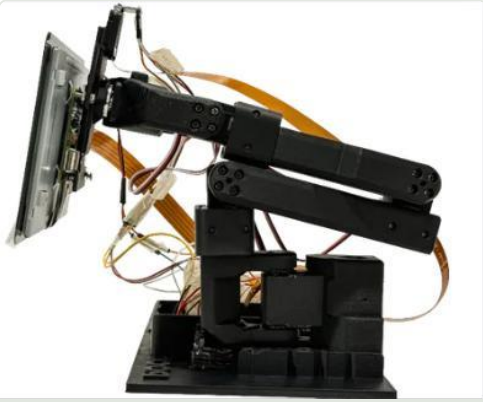
IMU 기반 짐벌 제어와 사용자 화면, 멀티프로세싱 구조를 구현했습니다. Git 형상 관리와 팀원별 코드 병합을 담당하고 통합 동작을 확인했습니다.

핵심 구현

- **IMU 기반 짐벌 제어**
  IMU 기울기와 PID 제어를 이용해 Motor 3·4의 각도를 보정하고 모니터 자세를 유지했습니다.
- **사용자 제어 · 분석 화면**
  PyQt 조작·상태 화면과 Streamlit·Plotly 자세 리포트를 구현했습니다.
- **병렬 처리 · 형상 관리 · 코드 통합**
  비전·하드웨어 프로세스를 분리하고 데이터 전달을 구성했습니다. Git 기반 변경 관리와 코드 병합으로 통합 실행을 관리했습니다.



POCO 시제품 · 정면



4축 모니터암 · 측면

모니터암 제어 담당 범위

Motor 3·4 · IMU 기반 짐벌 제어

전체 4축 모니터암 중 IMU 기반 기울기 보정을 담당했습니다.
현재 기울기와 목표 자세의 오차를 모터 제어에 반영했습니다.

UI·비전·하드웨어 통합

사용자 입력과 각 프로세스의 상태 데이터를 연결하고, 병합된 코드의 기능 간 연동을 확인했습니다.

01 **카메라 입력** 영상 프레임 수집	02 **AI 자세 판단** 장치 내 자세 분석	03 **모니터암 제어** 위치 조절 · 짐벌 보정	04 **사용자 화면 · 기록** 상태 확인 · 리포트

Python · Raspberry Pi 5 · IMU / PID · PyQt · Shared Memory · Git / GitHub

GitHub ↗ 개발 보고서 ↗ 시연 영상 ↗

POCO 짐벌·UI·멀티프로세싱 구현

IMU 자세 보정, 사용자 인터페이스, 병렬 실행 구조 및 Git 기반 코드 통합

IMU 기반 짐벌 제어 01

기울기 오차에 따른 Motor 3·4 각도 보정

IMU 기울기 → 목표 자세 오차 → PID · 모터 보정

IMU 측정값과 목표 자세의 오차를 계산하고 PID 제어로 Motor 3·4를 구동했습니다. 모니터암 동작 중 기울기 변화를 보정하도록 제어 루프를 구성했습니다.

사용자 제어·분석 화면 02

사용자 보정·실시간 상태·자세 리포트 제공

프로필 · 설정 | 보정 · 측정 | 실시간 상태 | 기록 · 리포트

PyQt로 프로필·보정·측정·상태 화면을 구성하고, 사용자 명령과 처리 결과를 연결했습니다. Streamlit·Plotly로 저장된 자세 기록을 시각화했습니다.

프로세스 분리·지연 개선 03

병렬 실행 및 최신 프레임 우선 소비 정책 적용

카메라 · 비전 ↔ 공유 메모리 · 큐 ↔ UI · 하드웨어

공유 메모리와 큐로 프로세스 간 데이터를 전달하고, 상태값과 제어 이벤트를 구분했습니다. 프레임 누적에 따른 응답 지연은 최신 프레임 우선 소비로 개선했습니다.

자세 판단 완료 시간 ~~약 1.35초~~ → **약 50 ms**

Git 형상 관리·코드 통합 04

파트별 변경 관리·병합·통합 동작 확인

파트별 변경 → 코드 병합 → 통합 실행 확인

Git/GitHub로 팀원별 변경 사항을 관리하고 코드를 병합했습니다. 프론트엔드·비전·하드웨어의 데이터 전달과 실행 흐름을 정리하고 통합 동작을 확인했습니다.

측정 조건: 개발완료보고서 p.13의 Pose 처리 경로 내부 측정. 프레임 누적(Overrun) 발생 상태와 최신 프레임 우선 소비 적용 후를 비교했습니다. 현재 POSE_ONLY 모드 기준이며 Face 피로도 분석은 비활성화 상태입니다.

2026.07.23 - 07.30 · 4인 팀

KICK-CAN · STM32 분산 제어 RC카

Bluetooth 조종기와 CAN 차량 네트워크를 결합한 4노드 임베디드 시스템

조종기·주행·센서·표시 기능을 4개 노드로 분산한 RC카입니다. 조종기는 Bluetooth로 주행 명령을 전달하고, 차량 내부 3개 노드는 CAN으로 데이터를 공유합니다.

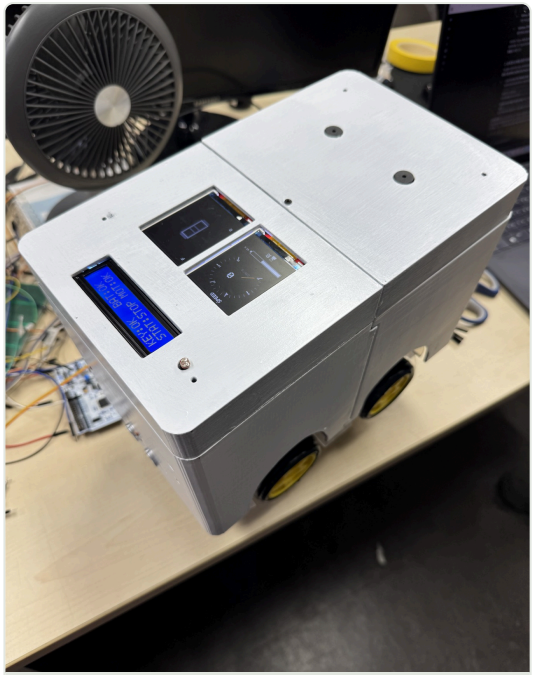
담당 역할

무선 조종기 하드웨어·펌웨어 개발

STM32F411 기반 조종기(Controller)를 제작하고, RFID 인증부터 입력 처리·제어 패킷 송신까지 펌웨어를 구현했습니다.

핵심 구현

- **RFID 인증 · 전원 인터록**
  등록된 RFID 카드 인증 후에만 주변장치 전원과 제어 패킷 송신을 허용했습니다.
- **조이스틱 · 스위치 입력 안정화**
  ADC DMA로 입력을 수집하고 EMA·Dead Zone·디바운싱으로 노이즈와 중복 입력을 줄였습니다.
- **제어 패킷 생성 · Bluetooth 송신**
  입력을 제어 패킷으로 변환하고 인증·전원 상태 확인 후 HC-05로 송신했습니다.



KICK-CAN 전체 차량



담당 제작물 · STM32 Controller

담당 노드 **Controller** 인증 · 입력 · 패킷 → Bluetooth UART / HC-05 → 차량 **Main Node** 4WD 주행 제어 ↔ CAN Bus ↔ 차량 **Node 1 · Node 2** 센서 · 표시 · 경고

STM32F411 · C / HAL · FreeRTOS · ADC / DMA · SPI · UART / Bluetooth

Controller GitHub ↗ 발표·시연 자료 ↗

STM32 조종기 인증·입력·무선 통신

RFID 기반 전원·송신 인터록, 입력 안정화, Bluetooth 제어 패킷 전송 구현

01 **RFID 인증** 카드 UID 확인 → 02 **주변장치 활성화** 릴레이 활성화 → 03 **입력 안정화** 조이스틱 · 스위치 → 04 **패킷 생성** 명령 ID · 데이터 → 05 **무선 송신** Main Node로 전달

RFID 인증 기반 전원·송신 인터록

인증 전

제어 잠금

STM32·RFID만 동작
주변장치 전원 OFF
제어 패킷 송신 차단

등록 카드 인증 후

제어 허용

릴레이로 주변장치 ON
입력 처리 시작
정상 상태에서 송신

SPI로 MFRC522의 카드 UID를 읽어 등록 사용자를 판별했습니다. 릴레이 전원 제어와 펌웨어 송신 차단을 결합해 인증 상태를 이중으로 확인했습니다.

> **인증 실패 처리**
>
> 인증 전에는 HC-05 전원이 차단되므로 미등록 카드 상태를 로컬 LED로 표시하고 잠금을 유지하도록 처리했습니다.

입력 안정화 및 통신 처리

조이스틱 입력 연속 수집

ADC Circular DMA로 2축 아날로그 입력을 메모리에 연속 저장해 수집과 처리를 분리했습니다.

조이스틱 중립 입력 안정화

EMA와 중심 보정으로 편차를 줄이고, Dead Zone과 Clamp로 중립 구간 및 출력 범위를 제한했습니다.

스위치 중복 입력 억제

GPIO EXTI에서는 이벤트만 전달하고, 일반 실행 흐름에서 디바운싱과 최종 상태 판정을 수행했습니다.

태스크 분리 · 상태 기반 송신

FreeRTOS로 인증·입력·송신 기능을 분리하고, 인증·전원 상태에 따라 UART 패킷 송신을 제어했습니다.

동작 검증 등록·미등록 카드에 따른 제어 허용 여부, 조이스틱 중립·방향 입력, 스위치 반복 조작을 확인했습니다. Main Node 연동을 통해 송신 명령의 차량 동작 반영을 검증했습니다.

PLC 기반 소재 분류·적재·배출

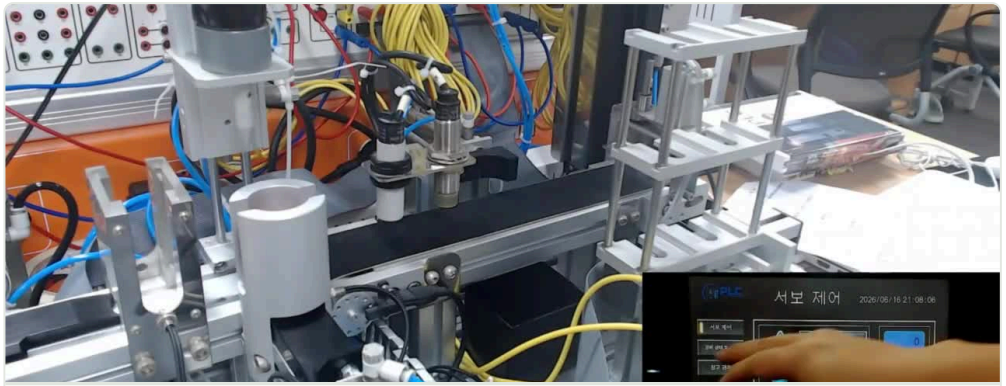
금속·비금속 분류, 위치별 적재, 자동·선택·전체 배출을 수행하는 자동화 공정

공급된 소재를 금속·비금속으로 분류해 6개 위치에 적재하고, 종류별 저장 위치를 관리하는 시스템입니다. 동일 소재 3개 적재 시 자동 배출하며, HMI에서 선택·전체 배출도 지시할 수 있습니다.

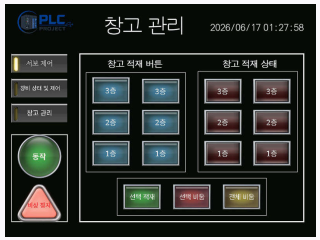
담당 역할

HMI 설계 · 배출 시퀀스 제어

적재 상태 표시와 배출 조작을 위한 HMI를 구성했습니다. 기본·선택·전체 배출 시퀀스와 명령 간 인터록을 PLC 로직으로 구현했습니다.



소재 이송·적재·배출 설비



담당 HMI · 적재 상태 표시 및 배출 제어

핵심 구현

- **적재 상태 표시 · 배출 조작 화면**
  위치별 소재 종류와 적재 상태를 표시하고, 선택 비움·전체 비움 명령을 PLC에 전달하도록 구성했습니다.
- **기본 · 선택 · 전체 배출**
  선택 배출은 기본 시퀀스 완료 후 실행하고, 전체 배출은 적재된 위치만 순차 처리했습니다.
- **서보 시퀀스 · 명령 인터록**
  서보 이동·흡착·배출 순서를 제어하고, 인터록으로 배출 명령의 중복 실행을 방지했습니다.

소재 공급 → 금속·비금속 분류 → 위치별 적재 → **배출 모드 선택** → **서보 이동 · 흡착 · 배출**

Mitsubishi PLC | GX Works2 | GT Designer3 | Ladder | Servo | Interlock

발표·시연 자료 ↗ 배출 제어 설명 ↗